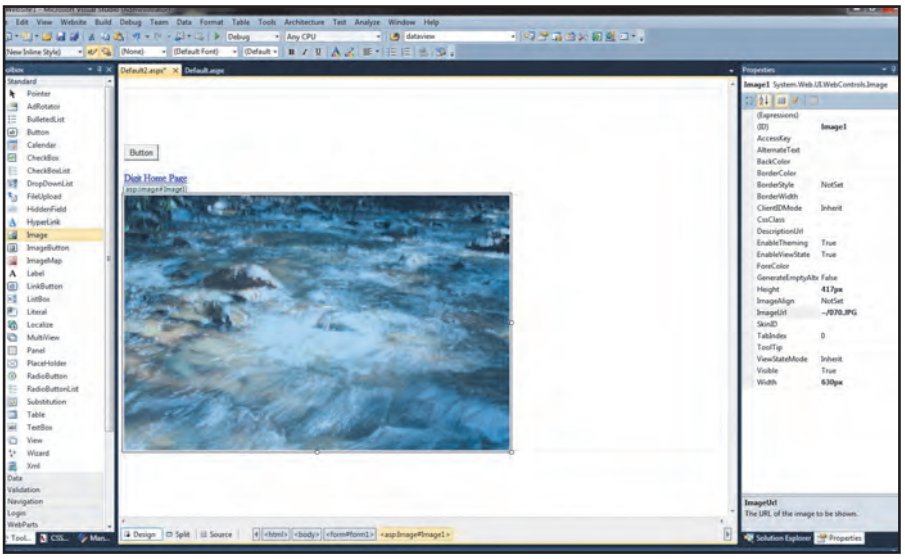
Properties window next to the Solution Explorer. There's more here than just setting the text, as you've done so far. Go to the Appearance section of the Properties, and play around with them to your liking. After you've kicked the tires a bit, run your application to see how it looks.

Visual Studio offers a number of different controls, to create lists from which the user can make a selection. These include the ListBox, the DropDownList, RadioButton and CheckBoxes. These controls work more or less the way you'd expect them to. There are several ways to display dynamically generated read-only text in your page — you can set the property of a label control, or you can use a read-only TextBox control.



Working with images is easy in Visual Studio

Adding images and links

An image can be a photograph, a drawing or even a logo. Visual Studio provides several controls to work with images.

`ImageControl` is used to display an image. `ImageButton` is used to create an image that can be clicked, acting like a normal button. An `ImageMap` control provides an image with multiple clickable hotspots. Each hotspot behaves like a separate hyperlink.

To insert an Image control into the form, drag a CheckBox and an Image control onto the form. In the Properties window, set the ID of the CheckBox to `cbDisplayPhoto`, and be sure to check AutoPostBack to `True`, and Text to `Display Image`. Also, set the TextAlign property to `Centre`.

Set the ID for the image to `Image1` and the ImageURL to the file you would use. Once you have the file, simply drag and drop it on to the Website1 folder in Solution Explorer. You'll see the image file appear in the file tree alongside your other solution files. You can also use any image file you have handy.

The Image control has three essential properties: the ID (so that you can address the control via code), the ubiquitous `runat="server"`, and the ImageUrl that identifies the location of the image in your directory. Because you put this image in the base directory of the application, you do not need a path name.

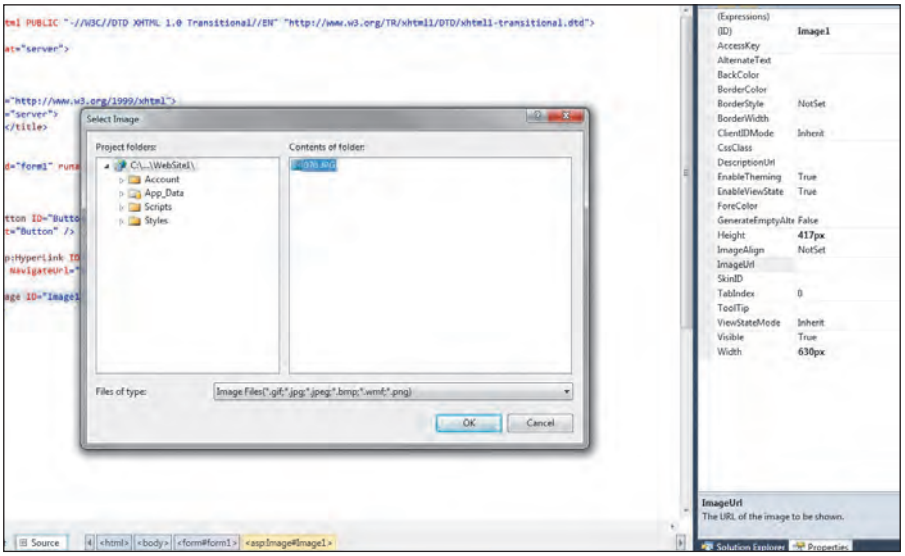
The name of the file itself will suffice. On our web page, the CheckBox control offers the user the opportunity to control the image visibility. It has its AutoPostBack property set to true so as to force a post-back every time the checked property changes. To make use of this, of course, you will have to write an event handler for the CheckedChanged event.

Double-click the CheckBox to create an event handler for CheckChanged, and add the following line of code:

```
Image1.Visible = cbDisplayPhoto.Checked;
```

This event handler changes the Visible property of the image to the checked property output. When the property is set to false, the image isn't rendered. When you uncheck the box, the page posts back, and the image vanishes.

Hyperlinks provide immediate redirection to another page in the application or to a location elsewhere on the internet. We'll use a HyperLink control to provide a link to Digit's home page, serving here very much the same function as an `<a>` tag would do in HTML. Drag a HyperLink control on to the bottom of your form. Set the ID to `hypContact`, its NavigateURL to `http://www.`



Browse to the image you want to place in your site and add the relevant information. Visual Studio will do the rest